

MemoryAgentBench results — FactConsolidation, 6K to 262K

Kaleidoscope on the FactConsolidation task of MemoryAgentBench (Hu, Wang and McAuley, ICLR 2026), at four history lengths, against keyword search, no memory, and the whole history pasted into the prompt. 800 questions per setup, and 200 for the whole-history setup, which was run at 262K only.

Read this before the numbers

Every setup uses the same reader: GPT-5.6 Luna, reasoning effort **high**, with a 10-token answer cap. The only thing that differs between setups is what reaches its prompt. So the comparison *within* this page is controlled for the reader. It is not controlled against published MemoryAgentBench results, which use other readers, and none are compared here.

Five further things, stated rather than adjusted for:

- **Kaleidoscope pays to write memory.** Before any question is asked, a model turns each piece of the history into a structured memory: 1,583 calls for the 262K history. That cost is paid once per history. Every cost on this page includes it, unless it says “reading only”.
- **The whole-history setup was scored by MemoryAgentBench itself**, like every other setup, on all 100 single-hop and all 100 multi-hop questions. `score_partial.py` recomputes both of its scores from the run’s log and gets the same numbers.
- **Each cell is 100 questions**, so a gap of a few points can be chance.
- **Not yet re-run from this repository.** These numbers came from this code before it moved here. The move changed file names, paths, imports and lint, not what it measures.
- **The dataset caps some scores.** FactConsolidation says the newest version of a fact wins, but scores some questions against an older one. That makes 2-3% of single-hop and 13-34% of multi-hop questions unanswerable by its own rule, for every setup.

Correction. An earlier version of this page compared only the tokens the answering model reads, and said Kaleidoscope used 244 times fewer tokens than the whole history. Counting the tokens it spends writing memory, it uses 6.3 times fewer over the 200 questions at 262K. That version also said the whole-history setup was not run on multi-hop questions and was not scored by MemoryAgentBench. It was run on both question types, MemoryAgentBench scored it, and it leads on multi-hop, 25 to Kaleidoscope’s 10.

At 262K tokens

Scores are out of 100. Tokens and cost cover all 200 questions at this length, 100 of each type, at list price: \$0.20 per million input tokens and \$1.20 per million output tokens.

	single-hop	multi-hop	total tokens	cost
entire history in the prompt	85	25	58.4M	\$11.71
Kaleidoscope (writing + reading)	90	10	9.3M	\$6.34
BM25 keyword search	78	11	0.46M	\$0.14

Accuracy. Kaleidoscope is the most accurate on single-hop questions: 90, against 85 for the whole history and 78 for BM25. On multi-hop questions the whole history leads with 25; Kaleidoscope scores 10 and BM25 11.

Cost against the whole history. Over these 200 questions, Kaleidoscope uses 6.3 times fewer tokens and costs 46% less. Most of its cost is writing: 9.1M tokens and \$6.25 to store the 262K history, paid once. After that, each question costs it about 1,300 tokens, against about 292,000 for the whole history. So the writing is repaid after about 31 questions in tokens, or about 107 in dollars. The two differ because about half of the writing tokens are output tokens, which cost six times as much as input tokens, while the whole history is almost all input.

Cost against BM25. BM25 is far cheaper than both, because it writes nothing: 0.46M tokens and \$0.14 for all 200 questions. That is about a twentieth of Kaleidoscope’s tokens and 2% of its cost. Kaleidoscope is 12 points more accurate on single-hop questions.

Reading only. These are the tokens sent to the answering model per single-hop question, which the earlier version of this page compared on their own:

reading tokens per single-hop question	
entire history in the prompt	291,746
Kaleidoscope	1,195
BM25 keyword search	2,066
no memory	189

For reading alone, Kaleidoscope sends 244 times fewer tokens than the whole history and 42% fewer than BM25. These figures leave out the writing.

The whole-history setup is not a ceiling. Its errors are confident wrong answers naming an *older* value of a fact that was later overwritten: shown every version, the reader sometimes picks a stale one. That is consistent with the errors; it has not been measured directly.

Accuracy at every length

Out of 100 questions per cell, single-hop / multi-hop:

history length	no memory	BM25	Kaleidoscope
6K	19 / 0	97 / 33	95 / 15
32K	26 / 0	91 / 13	95 / 8
64K	24 / 0	85 / 13	96 / 10
262K	13 / 2	78 / 11	90 / 10

As the history grows, keyword search falls from 97 to 78 on single-hop. Kaleidoscope holds between 95 and 90.

How often the answer was among the ten results each system considered:

history length	Kaleidoscope	BM25
6K	100%	96.9%
32K	100%	93.8%
64K	100%	86.9%
262K	98%	83.8%

Tokens at every length

Writing, which only Kaleidoscope does, once per history:

history length	model calls	tokens	cost
6K	39	241,643	\$0.18
32K	200	1,210,215	\$0.87
64K	399	2,367,626	\$1.67
262K	1,583	9,082,687	\$6.25

Writing grows roughly in step with the length of the history.

Reading only: input tokens sent to the answering model per single-hop question.

history length	no memory	BM25	Kaleidoscope	Kaleidoscope saves vs BM25, reading only
6K	189	1,879	854	55%
32K	190	1,999	948	53%
64K	189	2,007	1,019	49%
262K	189	2,066	1,195	42%

BM25 always sends ten chunks. Kaleidoscope sends what the question needs: on average 3.7 to 5.1 memories per single-hop question, depending on the length.

Where Kaleidoscope loses

Multi-hop, at every length. BM25 leads 33 to 15 at 6K and stays ahead at every longer length. At 262K the whole history leads both, with 25 to Kaleidoscope’s 10. This is a real loss, not noise.

At shorter histories the second fact in a chain is almost always retrievable, so the failure there is in which results are kept and in what order rather than whether the store can find them. At 262K, reaching it is itself the problem. A second retrieval keyed on the first answer closes over half the chains; under the same conditions, keyword search closes more of them.

An earlier analysis reported a much higher closure rate. That probe took its bridge from the answer key, which no retriever has at question time. It was an oracle and is withdrawn.

Cost, against keyword search. BM25 writes nothing, so at 262K it answered all 200 questions for \$0.14, against Kaleidoscope’s \$6.34.

Reproducing

Setup is in the benchmark’s README. Every number on this page comes from one of these:

number	where it comes from
scores at every length, both question types	<code>python -m kbench.benchmarks.memoryagentbench.run --run_id <id> --lengths 6k,32k,64k,262k</code>
the tables, how often the answer was retrieved, and memories sent per question	<code>python -m kbench.benchmarks.memoryagentbench.check <id> --out report.md</code>
writing and reading tokens and cost, per length, for Kaleidoscope, BM25 and no memory	the “Tokens and cost” section of that report
the whole-history setup	<code>python -m kbench.benchmarks.memoryagentbench.run_full_context --run_id <id2> --ingest_run_id <id></code>
its scores, 85 and 25, and its prompt tokens per question, 291,746 and 291,755	<code>python -m kbench.benchmarks.memoryagentbench.score_partial <parquet> <log> --row-kind sh</code> , then with the multi-hop log and <code>--row-kind mh</code>
its 58.4M tokens	those prompt tokens per question x 100 questions, for each type, plus 36,042 output tokens: <code>output_len</code> in MemoryAgentBench’s results file for each type is the average per question
every dollar figure	input tokens x \$0.20 per million + output tokens x \$1.20 per million

number	where it comes from
6.3 times fewer, 46% less, and the break-even points	the 262K table: the totals divided, and the writing cost divided by what each question saves against the whole history

<parquet> is `results/memoryagentbench/data/Conflict_Resolution-00000-of-00001.parquet`. <log> is `results/memoryagentbench/logs/mab_adapter-<id2>-full_context-factconsolidation_sh_262k.jsonl`, with `mh` in place of `sh` for multi-hop. MemoryAgentBench’s own results files for the whole-history setup are under `results/memoryagentbench/results/mab/<id2>/full_context/`, and hold the same two scores.